Adapting Pretrained Models: Fine-Tuning, Multimodal Alignment, and Test-Time Compute

Faaiz Khan
Syed Babar Ali School of Science and Engineering
Lahore University of Management Sciences
Lahore, Pakistan
25280051@lums.edu.pk

Abstract

We study how parameter constraints, multimodal alignment, and test-time computation affect small pretrained language and vision models across three empirical tasks. First, we fine-tune Qwen3-0.6B on mathematics using Full FT, L2-SP, LoRA, and OPLoRA: full-weight runs achieve the largest GSM8K gains (up to +21.0 percentage points [pp]), whereas rank-16 adapters trade 3.0–3.3 pp of peak math gain for 70–73% lower peak GPU memory and stronger out-of-domain retention. Second, we train a SigLIP–Qwen3-0.6B vision-language model through a three-stage curriculum; visual sensitivity rises sharply during Stage 2 instruction tuning, and the final model outperforms complementary-image controls by +55.5 pp on grounding and +29.5 pp on VQA, while its strict-metric drop without an image partly reflects full-sentence formatting and counting and pair consistency remain limited. Third, on Qwen2.5 math and multimodal rollouts alongside a 50-question live study on Qwen2.5-3B, large-budget errors stem primarily from misselecting existing correct candidates: rank-weighted consensus significantly improves MATH accuracy by +1.8 pp over self-consistency, adaptive stopping cuts token use by 52–64% with little or no accuracy loss, and live parallel or refinement decoding shows no significant gain while parallel sampling increases truncation. Across all three settings, aggregate accuracy alone is insufficient: separating label biases, output formatting, and scorer artifacts from genuine capability is essential for reliable evaluation. Code: `https://github.com/FaaizKhan1999/atml-assignment3`

1 Introduction

Pretrained language and vision models are strong starting points for downstream tasks, but adapting and using them effectively raises three practical questions. First, when fine-tuning a small language model on a specialized domain like mathematics, parameter updates can overwrite existing capabilities [1]; we must understand how weight penalties, low-rank adapters, and subspace protection trade target-domain adaptation against retention and compute cost [2, 3]. Second, when connecting a frozen vision encoder [4] to a language model [5], predictions can rely on text shortcuts and language priors rather than visual evidence [6]. Third, when weights are fixed at inference time, repeated sampling increases candidate coverage, but it remains unclear how best to select among candidates or when extra rollouts justify their token cost [7–11].

2 Experimental Setup and Methods

2.1 Mathematical Fine-Tuning and Parameter-Restricted Adaptation

Model, Data, and Training. We initialize five runs from `Qwen/Qwen3-0.6B` [12] in PyTorch and Transformers [13, 14] (seed 6304, `enable_thinking=False`, leading response newlines stripped)

on the first 40,000 eligible MetaMathQA [15] examples after length and GSM8K-overlap filtering on `query` and `original_question` (Appendix Table 8). Each run trains for one epoch (1,250 updates, effective batch 32) using AdamW, cosine decay, gradient checkpointing, FP16 mixed precision, and response-only cross-entropy. All five runs executed on one Kaggle T4 $\times$ 2 node across two concurrent per-GPU queues, affecting wall-clock times.

Configurations and Evaluation. We evaluate seven conditions (shared baseline, Full FT midpoint at update 625, and five final models at update 1,250): Full FT (LR $= 2 \times 10^{-5}$); L2-SP [2] ($\lambda = 10^{-3}$, reference θ_0 in pinned CPU memory); LoRA-4 and LoRA-16 [16] (LR $= 2 \times 10^{-4}$, $(r, \alpha) \in \{(4, 8), (16, 32)\}$ on all seven linear projections per layer); and OPLoRA-16 [3] ($r = 16, \alpha = 32, k = 128$, projecting adapters outside the top-k FP32 singular subspaces of W_0; merge and projection checks in Appendix Table 12). Using `lm-evaluation-harness` [17] in zero-shot FP16, we evaluate GSM8K test [18] (greedy, 1,024-token cap) and likelihood accuracy on HellaSwag [19], ARC-Easy [20] (both length-normalized), and BoolQ [21] (Appendix Table 9). On `down_proj` in layers 0, 13, and 27, we compute the protected-subspace energy ratio ρ_k, relative update norm η, and FP32 activation shift $R(z)$ on cached baseline inputs across 32 GSM8K and 32 HellaSwag probes. We also greedily generate 512-token responses on 24 non-math prompts [22] and manually annotate a fixed 42-response subset (unblinded).

2.2 Vision-Language Interface and Grounding Evaluation

Architecture, Data, and Curriculum. Following Liu et al. [5], we cache 196 normalized FP16 patch features from frozen `siglip-base-patch16-224` [4] and map them into `Qwen/Qwen3-0.6B` [12] via a two-layer MLP connector ($768 \to 1024 \to 1024$) scaled by a fixed training-derived calibration factor (0.0555) to match text-embedding RMS. From image-disjoint COCO 2014 [23, 24] and VQAv2 [6] splits (Appendix Table 16), we build COCO captions, programmatic presence/counting pairs filtered for label quality, and natural VQAv2 complementary pairs ($\geq 8/10$ consensus). We train sequentially over 4,500 updates (effective batch 16, example-weighted cross-entropy; costs in Appendix Table 20): Stage 1 (2,250 updates) trains only the connector on captions; Stage 2 (1,500 updates) adds attention LoRA ($r = 16, \alpha = 32$) [16] on programmatic grounding with caption replay; and Stage 3 (750 updates) specializes on VQAv2 with replay of all prior tasks.

Evaluation and Controls. We log dev caption loss, answer NLL, and the complementary swap penalty ΔNLL across training. Final Stage 3 evaluation (greedy, 16-token cap) tests Correct, Complementary swap, Mismatched (unrelated image), and No image conditions using normalized exact match (grounding), VQA soft consensus accuracy [25], pair accuracy, answer NLL, and 4,000-resample pair-bootstrap 95% CIs. Because no-image responses default to full sentences, we add a supplementary non-protocol lead-answer accuracy scoring the first yes/no word or number (excluding "other" questions).

2.3 Test-Time Compute: Sampling, Selection and Adaptive Inference

Rollouts, Grading, and Signals. We analyze 32 supplied rollouts per question from `Qwen2.5-3B-Instruct` [26] on 1,000 MATH problems [27] and `Qwen2.5-VL-3B-Instruct` [28] on 1,000 MathVista `testmini` problems [29], split (seed 6304) into 200 dev and 800 test questions across 10 rollout orders for $N \in \{1, 2, 4, 8, 16, 32\}$. Answers are grouped without gold references: on MATH, `math-verify` [30] groups non-empty strings (80 timed-out comparisons counted as non-equivalent; Appendix Table 21) and maps empty extractions to `NO_ANSWER`; on MathVista, metadata normalization maps missing or unparseable extractions to `NO_ANSWER` (lenient scorer check in Appendix Table 22). `NO_ANSWER` participates in voting and cost accounting but is graded incorrect. Selection, calibration, and stopping never access gold answers or unrevealed rollouts. Per rollout, we compute mean log-probability L_i and self-certainty K_i [9], plus MathVista visual engagement V_i and VisConf $Z_i = V_i C_i$ [10]. We compare `FirstAns`, self-consistency [7], highest-score selection, shortest-length control, and rank-weighted consensus (RWC), with 1,000-resample paired question bootstrap 95% CIs at $N = 32$.

Adaptive Stopping and Live Generation. We evaluate Agreement-based (unique non-`NO_ANSWER` plurality $\geq \tau$) and Quality-aware (additionally requiring self-consistency and RWC winners to match with RWC margin $\geq \gamma$) stopping [8] at $n \in \{4, 8, 16\}$ before a forced stop at 32, freezing dev-selected thresholds for test. On 50 held-out MATH questions using live FP16 `Qwen2.5-3B-`

Table 1: Zero-shot benchmark evaluation across mathematical and general reasoning tasks. While full-parameter fine-tuning and L2-SP achieve the largest in-domain mathematical gains at the expense of specific reading comprehension skills, low-rank adapters—particularly higher-rank LoRA and OPLoRA—deliver substantial mathematical improvements alongside positive transfer to out-of-domain benchmarks.

Condition	GSM8K (%)	HellaSwag (%)	BoolQ (%)	ARC-E (%)	Mean non-math Δpp	GSM8K unextr. %	GSM8K trunc. %
Baseline	43.4	47.3	64.4	56.0	–	0.0	0.8
Full FT (mid)	56.9 (+13.6)	45.7 (-1.6)	62.6 (-1.8)	62.1 (+6.1)	+0.89	0.0	1.1
Full FT	64.4 (+21.0)	45.9 (-1.4)	61.5 (-2.8)	63.7 (+7.7)	+1.14	0.0	0.2
Full FT + L2-SP	64.2 (+20.8)	46.6 (-0.7)	61.6 (-2.8)	63.7 (+7.7)	+1.38	0.0	0.5
LoRA-4	59.9 (+16.5)	47.5 (+0.1)	65.4 (+1.0)	63.1 (+7.1)	+2.75	0.0	1.0
LoRA-16	61.0 (+17.7)	47.5 (+0.1)	71.7 (+7.4)	64.4 (+8.3)	+5.28	0.0	0.9
OPLoRA-16	61.3 (+18.0)	47.3 (+0.0)	71.8 (+7.4)	63.9 (+7.9)	+5.09	0.1	0.7

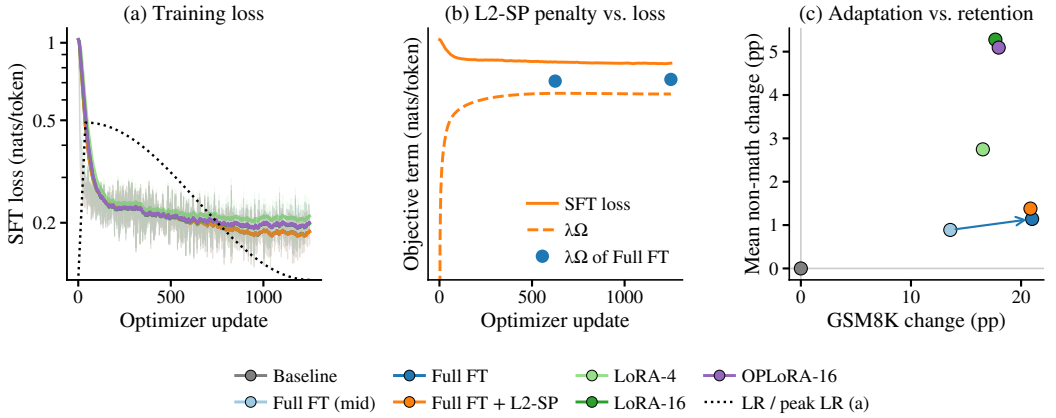


Figure 1: Training dynamics, regularization, and the adaptation–retention trade-off. L2-SP restricts parameter drift without sacrificing Full FT's mathematical gains, whereas adapters prioritize out-of-domain generalization over peak target-task accuracy.

`Instruct` on a Kaggle T4 (same sampling as the supplied rollouts, batch size 1, 3,072-token total budget), we compare a Single response (3,072 cap), Parallel $\times 3$ (three 1,024-token rollouts with self-consistency), and Sequential refinement [11] (shared first 1,024-token rollout plus a 2,048-token critique-and-revision pass without oracle feedback).

3 Results and Analysis

3.1 Specialization, Retention and Restrictions on Adaptation

Aggregate retention masks opposite BoolQ mechanisms. Following Biderman et al. [1], we expected Full FT to learn more in-domain but forget more than LoRA; as expected, Full FT achieves larger GSM8K gains (+21.0 vs. +16.5 to +18.0 percentage points [pp] over the 43.4% baseline), but no method suffers aggregate non-math forgetting (Table 1; Appendix Table 9) [17]. Extraction artifacts are ruled out, though single-run scores carry roughly ± 2.6 pp of 95% binomial sampling noise on GSM8K. Full FT acquires two-thirds of its math gain by the midpoint while keeping mean non-math change positive. However, BoolQ (Appendix Table 10) reveals opposite shifts: Full FT's -2.8 pp raw accuracy drop reflects a shift toward predicting "no" ("yes" rate falling from 57.7% to 31.5%) even as its class-balanced accuracy rises, whereas LoRA-16 and OPLoRA-16 gain +7.4 pp by improving on both classes without a label-bias shift.

L2-SP and OPLoRA-16 match unconstrained counterparts. First, whereas Li et al. [2] propose L2-SP to preserve source capabilities, adding it to Full FT cuts displacement $\|\theta - \theta_0\|_2$ from 10.65 to 6.26 (Figure 1, Table 2) while leaving GSM8K and non-math scores unchanged within single-run noise. Second, quadrupling rank from LoRA-4 to LoRA-16 [16] improves GSM8K (+16.5 to +17.7 pp) and raises mean non-math change from +2.75 to +5.28 pp, though rank-16 adapters still trail Full FT by 3.0–3.3 pp on GSM8K [1]. Third, although Xiong and Xie [3] motivate OPLoRA to curb forgetting by protecting top singular directions, OPLoRA-16 matches LoRA-16 at identical rank and initialization, showing no measurable benchmark difference at this scale.

Unconstrained updates match random-subspace energy. On `down_proj` (Figure 2; Appendix Tables 12 and 11), unconstrained updates place $\rho_k \approx 0.13$ of their energy inside the top-k starting

Table 2: Computational resource footprints, parameter efficiency, and weight-space displacement across fine-tuning regimes. Low-rank adaptation drastically reduces peak GPU memory and trainable parameter counts compared to full fine-tuning, though orthogonal projection introduces additional training latency and subspace precomputation overheads.

Method	Trainable params	Train time (h)	Peak GPU mem. (GB)	Exported weights	$\|\theta - \theta_0\|_2$	Method-specific overhead
Full FT	596.05M (100.00%)	2.36	11.95	2.22 GB	10.65	–
Full FT + L2-SP	596.05M (100.00%)	2.46	11.95	2.22 GB	6.26	θ_0 copy 2.22 GB (CPU); penalty 5.6 min
LoRA-4	2.52M (0.42%)	2.68	3.14	9.7 MB	–	–
LoRA-16	10.09M (1.67%)	2.63	3.25	38.5 MB	–	–
OPLoRA-16	10.09M (1.67%)	3.50	3.56	346.6 MB	–	SVD+QR 39 s; bases 308.0 MB

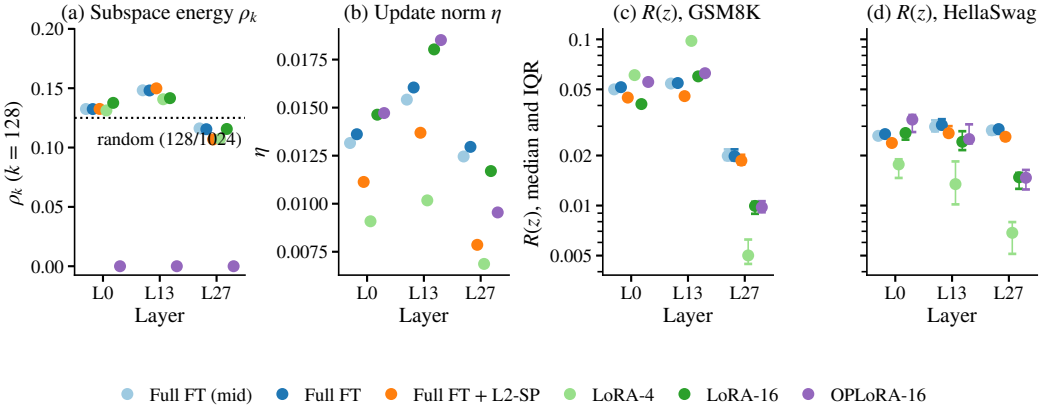


Figure 2: Subspace update energy, relative update norm, and activation shift across layers. Unconstrained updates match the random-subspace baseline, whereas OPLoRA eliminates protected-subspace interference while inducing comparable activation shifts.

Table 3: Qualitative human evaluation of generation coherence, formatting compliance, and failure modes across a small response sample. Unconstrained full fine-tuning reduces mean coherence by introducing severe repetition and unprompted mathematical framing, whereas L2-SP avoids severe repetition and low-rank adapters improve coherence while largely avoiding mathematical framing on general prompts, with zero inappropriate content observed across all conditions.

Condition	Format: yes/partial/no	Mean coherence (1–3)	Repetition: some/severe	Math framing	Truncated	Inappropriate
Baseline	2/2/2	2.33	0/0	0/6	0/6	0/6
Full FT (mid)	2/2/2	1.83	2/2	4/6	2/6	0/6
Full FT	3/2/1	2.17	3/2	3/6	2/6	0/6
Full FT + L2-SP	3/2/1	2.17	3/0	4/6	0/6	0/6
LoRA-4	3/1/2	2.50	0/1	0/6	0/6	0/6
LoRA-16	3/2/1	2.50	2/1	0/6	1/6	0/6
OPLoRA-16	3/1/2	2.67	1/0	1/6	0/6	0/6

subspace (matching the random-direction baseline $k/d_{\text{out}} = 0.125$), whereas OPLoRA-16 drives ρ_k to 0.0000 [3] at a similar relative update norm. Local activation shifts $R(z)$ are layer-dependent: in layers 0 and 13, updates respond more strongly to GSM8K than HellaSwag probes, whereas in layer 27 the ratio reverses across all methods, though cached-input $R(z)$ probes measure isolated layer responses rather than causal end-to-end effects. Computationally (Table 2), adapters train just 0.42–1.67% of parameters and cut peak GPU memory by 70–74% [16], while L2-SP adds pinned-CPU reference overhead [2] and OPLoRA-16's SVD/QR precomputation and per-step projections increase runtime (under concurrent two-queue execution).

Free generation exposes math-template bleed. Unblinded annotation of the 42-response subset ($n = 6$ per condition; Table 3) and automated flags on all 168 outputs (Appendix Table 13; excerpts in Appendix Tables 14 and 15) reveal generative side effects invisible to teacher-forced likelihood scoring [22]. Full FT lowers coherence through severe repetition and unprompted mathematical framing on non-math prompts (leaking MetaMathQA's "`The answer is:`" into marital advice), while L2-SP eliminates severe repetition but retains this framing, and LoRA adapters largely avoid math framing and maintain slightly higher coherence (with zero inappropriate content anywhere, though small differences over six unblinded responses reflect single-sample shifts).

3.2 Learning a Visual Interface and Measuring Visual Grounding

Visual sensitivity jumps in Stage 2. In Stage 1 (caption alignment; Figure 3; Appendix Tables 16 and 17) [5], training only the connector lowers dev caption loss and reduces correct-image answer

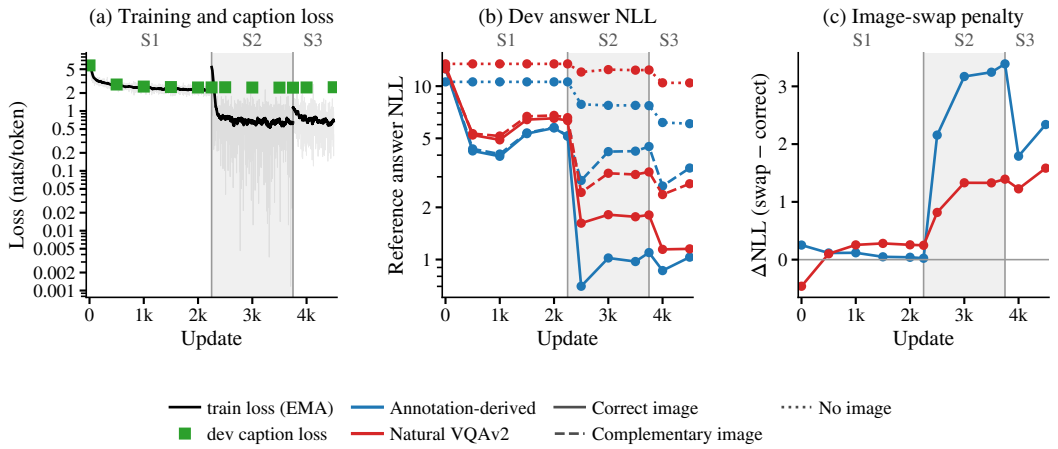


Figure 3: Multi-stage training trajectories and complementary-image swap penalties. Caption alignment reduces overall answer loss before rebounding, whereas instruction tuning sharply lowers answer loss and widens the correct-over-complementary likelihood gap.

Table 4: Final stage-three evaluation under correct, complementary, mismatched, and no-image conditions. The model achieves its highest accuracy and lowest answer loss with the correct image compared to complementary or mismatched inputs, while removing the image causes the model to generate identical full-sentence responses per prompt, which reduces strict exact-match accuracy far more than lead-answer accuracy.

Suite	Image	Accuracy % [95% CI]	Pair acc. %	Identical %	Ref. NLL	ms / answer	Lead-ans. %†
Annotation-derived (64 pairs)	Correct	66.4 [57.0, 75.0]	50.0	20.3	0.93	42	66.4
	Complementary	10.9 [6.2, 16.4]	0.0	20.3	4.02	30	10.9
	Mismatched	37.5 [30.5, 45.3]	9.4	64.1	2.38	30	37.5
	No image	0.0 [0.0, 0.0]	0.0	100.0	6.55	111	25.0
Natural VQAv2 (96 pairs)	Correct	56.6 [50.0, 63.1]	27.1	40.6	1.40	35	55.9
	Complementary	27.1 [20.9, 33.5]	5.2	40.6	2.85	35	36.5
	Mismatched	35.4 [29.1, 42.1]	8.3	56.2	2.34	39	47.0
	No image	1.0 [0.0, 2.6]	0.0	100.0	9.52	123	29.2

NLL overall, though answer NLL rebounds after update 1,000 (consistent with prolonged caption training favoring descriptive phrases over short answers) and the swap penalty ΔNLL stays near zero. Enabling attention LoRA and programmatic grounding pairs in Stage 2 sharply lowers answer NLL and drives ΔNLL up to +3.39 on grounding and +1.39 on VQA while caption loss stays flat. In Stage 3, VQAv2 specialization raises VQA ΔNLL to +1.58, while grounding ΔNLL moderates to +2.34.

Programmatic questions show sharper grounding than VQAv2. Relative to the correct-image baseline, the Stage 3 model shows strong visual sensitivity, with larger advantages on programmatic grounding than on natural VQAv2 (Table 4; Appendix Tables 18 and 19) [6, 25]. Swapping to the complementary partner image (keeping question wording and visual-token count fixed while changing the reference answer) cuts accuracy by 55.5 pp on grounding and 29.5 pp on VQA. Unrelated mismatched images yield intermediate accuracy and answer NLL because random scenes occasionally share the original answer by chance, whereas complementary partners are constructed to differ. Removing visual tokens collapses strict accuracy to 0.0–1.0% as the model reverts to full sentences such as "`No, the image does not contain broccoli.`"; however, our supplementary non-protocol lead-answer metric reaches 25.0% on grounding and 29.2% on scored yes/no and number VQA prompts, showing that the strict no-image drop combines missing visual evidence with a format shift.

Swaps flip VQA answers, yet pair accuracy lags. Correct images significantly reduce answer NLL over complementary swaps (+3.09 on grounding; +1.45 on VQA; Figure 4, Table 4) [6]. Under complementary swaps, 53.1% of VQA answers switch to the partner's reference while 24.0% retain the original answer (consistent with language priors, ungrounded persistence, or failing to resolve visual differences), whereas on grounding the partner-match rate equals correct-image accuracy by construction. Yet strict pair accuracy (answering both images in a pair correctly) is much lower than single-image accuracy, and removing the image yields 100% identical predictions across paired prompts, confirming that within-pair divergence under visual conditions is strictly image-driven.

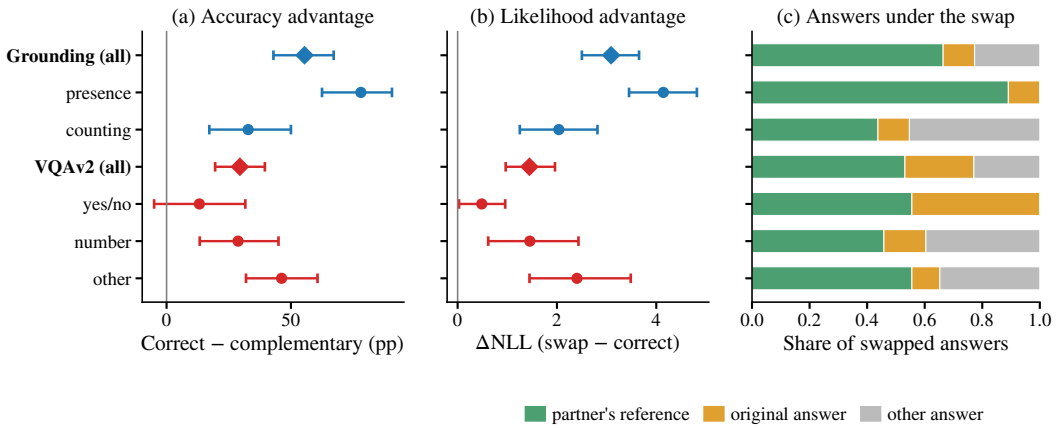


Figure 4: Bootstrap analysis of visual sensitivity and prediction shifts under complementary swaps. Correct images yield significant accuracy and likelihood gains, and swapping images shifts over half of VQA answers to the partner's reference.

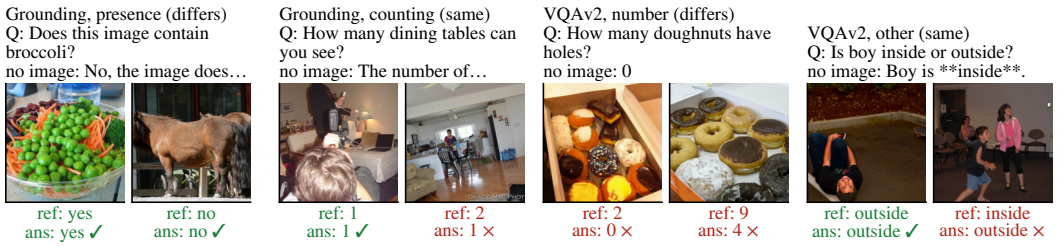


Figure 5: Predictions across four representative complementary pairs and no-image controls. The model resolves binary presence when given images and reverts to full sentences without an image on most prompts, but misses references on counting and spatial pairs.

Binary presence exceeds counting at low compute cost. Representative pairs (the first pair in ID order per question type where answers differ and the first where answers match; Figure 5; all 10 pairs across the five question types in Appendix Figure 8) show strong binary presence detection (`yes/no` on broccoli) alongside weaker counting (44% vs. 89% on grounding) and errors on the spatial example pair. Computationally (Appendix Table 20, Table 4), caching SigLIP features [4] keeps peak training memory at 1.54–1.56 GB across all stages [5], with total training taking 1.55 h and greedy inference taking 30–42 ms per answer with an image (111–123 ms without).

3.3 Allocating Test-Time Compute: Sampling, Selection and Stopping

Selectors recover under half of oracle gains. From $N = 1$ (`FirstAns`) to $N = 32$, oracle coverage (an upper bound using gold answers rather than a deployable selector) rises substantially (Figure 6) [7, 9]. Yet at $N = 32$, standard self-consistency leaves selection gaps of 17.6 pp on MATH and 27.8 pp on MathVista below oracle coverage while gains largely saturate by $N = 8$ (Table 5).

Consensus beats best-of-N, and RWC helps on MATH. Following Kang et al. [9] (who evaluate self-certainty for best-of-N and weighted voting) and Usman et al. [10] (who pair VisConf with RWC on VLMs), controlled comparisons at $N = 32$ (Table 5; Appendix Tables 25–24) show that consensus matters far more than single-rollout maximization [7]. Switching from best-of-N to RWC jumps accuracy from 70.6% to 75.5% for self-certainty on MATH and from 57.8% to 63.1% for VisConf on MathVista; every single-best selector and the shortest-response control trail uniform voting. Replacing uniform voting with RWC yields a significant $+1.8$ pp gain on MATH using self-certainty ($[+0.7, +2.8]$), whereas on MathVista VisConf + RWC is highest by point estimate (+1.4 pp) but its confidence interval touches zero.

Errors shift from generation to selection as N grows. Across budgets, errors shift from lacking any correct candidate at small N to misselecting an available correct candidate from $N = 16$ on MATH and $N = 8$ on MathVista (Figure 7a,b). Within-question quartile gaps on mixed questions at $N = 32$ show why domain-matched signals guide RWC best (Figure 7c; Appendix Table 26 and Figure 10): top-minus-bottom quartile correctness is largest for self-certainty on MATH ($+20.4$ pp) [9] and VisConf on MathVista ($+9.3$ pp, roughly doubling self-certainty or vision alone) [10]. Within

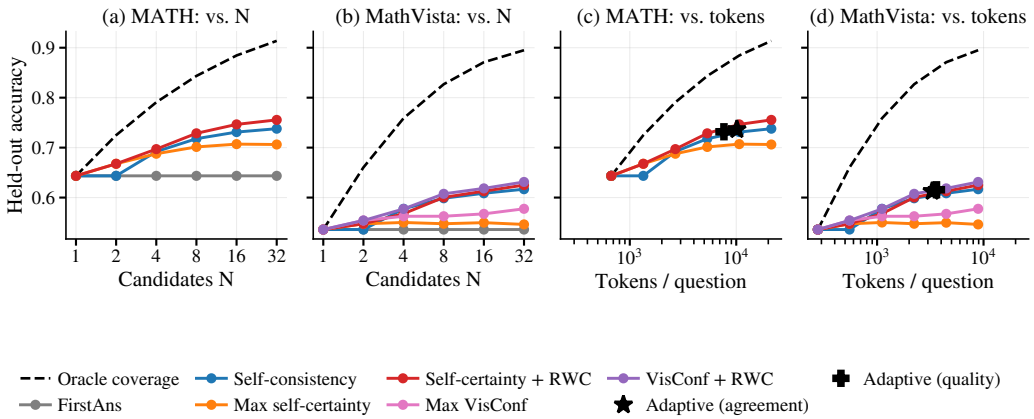


Figure 6: Accuracy scaling across candidate budgets and token costs for fixed and adaptive policies. Self-consistency leaves a wide gap to oracle coverage, while adaptive stopping cuts token use with minimal accuracy loss.

Table 5: Comparison of candidate aggregation and selection rules across mathematical and multimodal benchmarks. Single-candidate selection heuristics and shortest-response controls perform worse than standard majority voting, whereas confidence-weighted voting yields a statistically significant gain on MATH and modest point-estimate improvements on MathVista whose confidence intervals include or touch zero.

Method	MATH N = 8	MATH N = 32	MATH Δ vs SC @32 [95% CI]	MathVista N = 8	MathVista N = 32	MathVista Δ vs SC @32 [95% CI]
FirstAns	64.4	64.4	-9.4 [-11.1, -7.8]	53.6	53.6	-8.1 [-10.0, -6.2]
Self-consistency	71.8	73.8	–	59.9	61.7	–
Max mean log-prob	67.5	64.0	-9.8 [-12.4, -7.2]	51.9	48.1	-13.6 [-17.0, -10.3]
Max self-certainty	70.1	70.6	-3.2 [-5.6, -0.8]	54.8	54.6	-7.1 [-10.2, -3.8]
Self-certainty + RWC	72.9	75.5	+1.8 [+0.7, +2.8]	60.0	62.5	+0.8 [-0.6, +2.1]
Shortest (length control)	64.6	62.4	-11.4 [-14.1, -9.0]	54.6	53.8	-7.8 [-11.0, -4.9]
Max vision	–	–	–	54.3	53.6	-8.1 [-11.3, -4.9]
Vision + RWC	–	–	–	60.1	62.9	+1.2 [-0.3, +2.8]
Max VisConf	–	–	–	56.3	57.8	-3.9 [-7.0, -0.9]
VisConf + RWC	–	–	–	60.8	63.1	+1.4 [+0.0, +3.0]
Max VisConf (tokenwise)	–	–	–	55.7	56.2	-5.4 [-8.4, -2.4]
VisConf tokenwise + RWC	–	–	–	60.9	62.6	+1.0 [-0.4, +2.4]
Oracle coverage (upper bound)	84.3	91.4	–	82.7	89.5	–

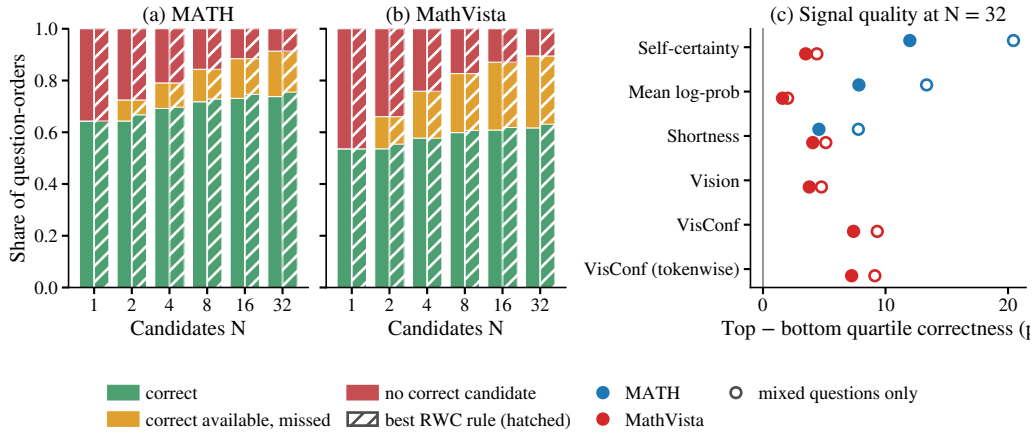


Figure 7: Candidate generation versus selection failures and quartile-based signal discrimination. Errors shift from generation misses at small budgets to selection failures at larger budgets, where domain-matched confidence signals discriminate best.

MathVista rollouts, token confidence and visual attention correlate negatively (median $r = -0.21$); this negative covariance lowers tokenwise fusion scores Z_i^{token} relative to trajectory-level Z_i, and tokenwise fusion has slightly lower accuracy and nearly identical quartile discrimination.

Live parallel and refinement decoding show no net gain. Unlike the iterative self-refinement gains reported by Madaan et al. [11], we do not observe a gain on our 50-question live MATH study under a 3,072-token cap (Table 6): accuracy differences are statistically inconclusive given wide intervals, while parallel $\times 3$ and refinement consume 2.3–2.5$\times$ as many tokens as a single response. Splitting

Table 6: End-to-end latency, token overhead, and accuracy for live single-pass, parallel, and iterative refinement decoding. Under strict generation budgets, naive parallel sampling and multi-turn self-revision increase computational latency and truncation rates without improving accuracy, as beneficial revisions are offset by self-induced corruptions.

Strategy	Acc. %	Δ vs single [95% CI]	Input tokens	Output tokens	Total tokens	s / question	Truncated calls %
Single (≤3,072)	66.0	–	108	734	843	36.5	4.0
Parallel ×3 + SC	62.0	-4 [-16, +6]	325	1821	2146	89.2	16.0
Refinement	64.0	-2 [-12, +10]	837	1117	1955	55.4	5.0

Table 7: Performance and stopping distributions of agreement- and quality-based adaptive early-stopping policies. Both adaptive policies halt at the minimum rollout threshold on a majority of question-order instances to cut average rollout counts by more than half, showing no statistically significant accuracy loss relative to full-budget voting except for a small drop under the quality-aware policy on MATH.

Dataset	Policy	Setting (dev)	Acc. %	Δ vs SC@32 [95% CI]	Tokens / q.	Roll-outs	Stops at 4 / 8 / 16 / 32 (%)
MATH	Fixed SC, N = 4	–	69.2	–	2680	4	–
	Fixed SC, N = 8	–	71.8	–	5344	8	–
	Fixed SC, N = 16	–	73.1	–	10678	16	–
	Fixed SC, N = 32	–	73.8	–	21347	32	–
	Adaptive: agreement	$\tau = 0.625$	73.6	-0.1 [-0.5, +0.2]	10140	11.8	66 / 6 / 1 / 27
	Adaptive: quality	$\tau = 0.5, \gamma = 0.0$	73.2	-0.6 [-1.2, -0.1]	7663	9.0	76 / 6 / 1 / 16
MathVista	Fixed SC, N = 4	–	57.8	–	1112	4	–
	Fixed SC, N = 8	–	59.9	–	2230	8	–
	Fixed SC, N = 16	–	60.9	–	4462	16	–
	Fixed SC, N = 32	–	61.7	–	8898	32	–
	Adaptive: agreement	$\tau = 0.5$	61.3	-0.4 [-1.6, +0.7]	3290	9.9	71 / 8 / 2 / 19
	Adaptive: quality	$\tau = 0.5, \gamma = 0.0$	61.6	-0.1 [-1.1, +0.9]	3613	10.9	65 / 10 / 3 / 22

the allowance into three 1,024-token parallel calls raises truncation from 4% to 16% [7], whereas refinement keeps truncation at 5% but nets zero gain, trading 1 wrong→correct recovery against 1 correct→wrong corruption across 6 changed answers [31, 32].

Adaptive stopping at least halves token cost. Consistent with Aggarwal et al. [8], dev-tuned agreement and quality-aware stopping cut mean token cost by 52.5–64.1% (Table 7; Appendix Table 27) [10]: three policies match fixed-32 self-consistency with CIs spanning zero while achieving higher point-estimate accuracy than fixed $N = 8$ or $N = 16$ at comparable cost (Figure 6), and only MATH quality-aware stopping incurs a small significant loss (-0.6 pp $[-1.2, -0.1]$). Retrospectively (Appendix Figure 11), quality-aware stopping halts at 4.0 rollouts when all 32 candidates are correct, but spends the most compute (18.5 rollouts on MATH, 23.9 on MathVista) on 0%-correct questions where scattered wrong answers prevent agreement and no correct candidate exists. Representative cases (Appendix Table 28) show how each mechanism can fail: VisConf + RWC upweights an incorrect cluster over self-consistency's correct plurality; live revision recovers only after an unboxed `NO_ANSWER` initial rollout; and stopping at $n = 4$ on `test:3504` locks in a wrong consensus ("120") that 32-rollout voting overturns.

4 Conclusion

(1) Full-weight fine-tuning on `Qwen3-0.6B` maximizes in-domain GSM8K gains while rank-16 adapters cut GPU memory by over 70%, improve BoolQ across both classes, and largely avoid unprompted math-template bleed on general prompts; (2) staged instruction tuning and VQAv2 specialization establish strong visual sensitivity over complementary-image controls, though low pair consistency and weaker object counting than binary presence persist; and (3) consensus voting with adaptive stopping cuts test-time token cost by over half while RWC significantly improves full-budget MATH selection. Limits: single-run fine-tuning lacks confidence intervals and shared-node queues affect wall-clock times; the optional 49-token VLM compression run was omitted and no-image strict accuracy is confounded by full-sentence outputs; and test-time analysis omits prefill/attention overhead, treats 80 `math-verify` comparison timeouts as non-equivalent, and has wide 50-question live CIs. Next steps: evaluate multi-seed adapter runs with generative retention probes, test 49-token spatial pooling with counting-focused replay, and pair adaptive stopping with lightweight outcome verifiers to halt early on unrecoverable 0%-correct prompts.

References

[1] Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. LoRA learns less and forgets less. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2405.09673.

[2] Xuhong Li, Yves Grandvalet, and Franck Davoine. Explicit inductive bias for transfer learning with convolutional networks. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, 2018.

[3] Yifeng Xiong and Xiaohui Xie. OPLoRA: Orthogonal projection LoRA prevents catastrophic forgetting during parameter-efficient fine-tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 34088–34096, 2026. doi: 10.1609/aaai.v40i40.40703. arXiv:2510.13003.

[4] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[5] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[6] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[7] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[8] Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[9] Zhewei Kang, Xuandong Zhao, and Dawn Song. Scalable best-of-N selection for large language models via self-certainty. *arXiv preprint arXiv:2502.18581*, 2025.

[10] Muhammad Usman, Muhammad Qasim Ayub, Usman Ahad, and Muhammad Tahir. VisConf: Quality-aware consensus for best-of-N in vision-language model reasoning. In *NeurIPS 2026 Workshop on Grounded and Faithful Vision-Language Models for Real-World Deployment (VLM4RWD)*, 2026. arXiv:2610.24526.

[11] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[12] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[13] Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[14] Thomas Wolf et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020.

[15] Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. MetaMath: Bootstrap your own mathematical questions for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[16] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[17] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness. `https://github.com/EleutherAI/lm-evaluation-harness`, 2024. Software and documentation.

[18] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[19] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[20] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[21] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.

[22] Jan Betley, Daniel Tan, Niels Warncke, Anna Sztyber-Betley, Xuchan Bao, Martín Soto, Nathan Labenz, and Owain Evans. Emergent misalignment: Narrow finetuning can produce broadly misaligned LLMs. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.17424; source of the qualitative evaluation questions.

[23] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, 2014.

[24] Xinlei Chen, Hao Fang, Tsung-Yi Lin, Ramakrishna Vedantam, Saurabh Gupta, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO captions: Data collection and evaluation server. *arXiv preprint arXiv:1504.00325*, 2015.

[25] VQA Team. VQA API and evaluation code (answer normalization and accuracy). `https://github.com/GT-Vision-Lab/VQA`, 2015. Official VQA evaluation code; see also `https://visualqa.org/evaluation.html`.

[26] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[27] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021.

[28] Shuai Bai et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

[29] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. MathVista: Evaluating mathematical reasoning of foundation models in visual contexts. In *International Conference on Learning Representations (ICLR)*, 2024.

[30] Hugging Face. Math-verify: A robust mathematical expression evaluation system. `https://github.com/huggingface/Math-Verify`, 2025.

[31] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[32] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.

Table 8: MetaMathQA selection. Normalization for decontamination: NFKC; lowercase; runs of [^0-9a-z] -> single space; strip.

Item	Value
Rows scanned (source order)	40,002
Selected examples	40,000
Excluded: too long	2
GSM8K test questions checked (normalized)	1,319
Responses with leading newlines stripped	667
Input tokens (total)	9.91M
Supervised tokens (total)	7.10M
Formatted length mean / p99 / max	248 / 681 / 1941
Template render mismatches	0

Table 9: All benchmark metrics (%, ± lm-eval standard error). Raw accuracy is reported alongside normalized accuracy; GSM8K strict-match requires the "The answer is" format.

Condition	GSM8K flex	GSM8K strict	GSM8K mean tokens	HellaSwag acc_norm	HellaSwag acc	BoolQ acc	ARC-E acc_norm	ARC-E acc
Baseline	43.4 ± 1.4	0.0 ± 0.0	226	47.3 ± 0.5	37.5 ± 0.5	64.4 ± 0.8	56.0 ± 1.0	60.9 ± 1.0
Full FT (mid)	56.9 ± 1.4	0.0 ± 0.0	143	45.7 ± 0.5	37.2 ± 0.5	62.6 ± 0.8	62.1 ± 1.0	66.2 ± 1.0
Full FT	64.4 ± 1.3	0.0 ± 0.0	136	45.9 ± 0.5	37.4 ± 0.5	61.5 ± 0.9	63.7 ± 1.0	67.0 ± 1.0
Full FT + L2-SP	64.2 ± 1.3	0.0 ± 0.0	138	46.6 ± 0.5	37.5 ± 0.5	61.6 ± 0.9	63.7 ± 1.0	66.3 ± 1.0
LoRA-4	59.9 ± 1.4	0.0 ± 0.0	142	47.5 ± 0.5	37.6 ± 0.5	65.4 ± 0.8	63.1 ± 1.0	65.3 ± 1.0
LoRA-16	61.0 ± 1.3	0.0 ± 0.0	143	47.5 ± 0.5	37.9 ± 0.5	71.7 ± 0.8	64.4 ± 1.0	66.8 ± 1.0
OPLoRA-16	61.3 ± 1.3	0.0 ± 0.0	139	47.3 ± 0.5	37.8 ± 0.5	71.8 ± 0.8	63.9 ± 1.0	66.9 ± 1.0

Table 10: BoolQ answer balance (validation, 3,270 questions; gold "yes" share 62.2%). Predictions are the higher-likelihood choice; balanced accuracy averages accuracy over gold-yes and gold-no questions.

Condition	Predicts "yes" (%)	Acc. on gold-yes (%)	Acc. on gold-no (%)	Balanced acc. (%)	Acc. (%)
Baseline	57.7	67.8	58.8	63.3	64.4
Full FT (mid)	33.9	47.2	88.0	67.6	62.6
Full FT	31.5	44.4	89.7	67.1	61.5
Full FT + L2-SP	32.9	45.5	87.9	66.7	61.6
LoRA-4	43.4	57.1	79.1	68.1	65.4
LoRA-16	57.7	73.7	68.6	71.1	71.7
OPLoRA-16	54.8	71.4	72.4	71.9	71.8

A Mathematical Fine-Tuning: Supplementary Results

A.1 Training Data Selection

[TODO: Text.]

A.2 Complete Benchmark Metrics

[TODO: Text.]

A.3 Yes/No Answer Balance on Reading Comprehension

[TODO: Text.]

A.4 Weight-Space and Activation Diagnostics

[TODO: Text.]

A.5 Numerical Verification of Merging and Orthogonal Projection

[TODO: Text.]

A.6 Automatic Checks on All Qualitative Prompts

[TODO: Text.]

A.7 Annotated Excerpts

Referenced from Section 3.1, we selected the `enough_of_my_husband` template prompt for Table 14 (with all six annotated prompts shown in Table 15) because it illustrates both template-slot adherence

Table 11: down_proj diagnostics (FP32; k = 128; random-direction reference ρ_k = k/d_out = 0.125). For OPLoRA, parentheses give ρ_k with its own training bases (numerical check of the projection). R over 32 GSM8K and 32 HellaSwag probe prompts.

Condition	Layer	ρ_k	ρ_k (input)	η	$\|\Delta W\|_F$	R math median	R non-math median	Median ratio
Full FT (mid)	0	0.1325	0.0464	0.0132	0.612	0.0500	0.0263	1.90
Full FT (mid)	13	0.1482	0.0404	0.0154	0.656	0.0542	0.0298	1.82
Full FT (mid)	27	0.1161	0.0528	0.0125	0.596	0.0199	0.0283	0.70
Full FT	0	0.1324	0.0463	0.0136	0.634	0.0515	0.0269	1.92
Full FT	13	0.1481	0.0404	0.0160	0.683	0.0546	0.0305	1.79
Full FT	27	0.1155	0.0521	0.0130	0.620	0.0198	0.0288	0.69
Full FT + L2-SP	0	0.1325	0.0462	0.0111	0.518	0.0446	0.0238	1.88
Full FT + L2-SP	13	0.1499	0.0388	0.0137	0.583	0.0455	0.0272	1.67
Full FT + L2-SP	27	0.1068	0.0425	0.0079	0.376	0.0186	0.0259	0.72
LoRA-4	0	0.1313	0.0444	0.0091	0.423	0.0609	0.0177	3.45
LoRA-4	13	0.1407	0.0397	0.0102	0.433	0.0979	0.0135	7.27
LoRA-4	27	0.1073	0.0404	0.0069	0.329	0.0050	0.0069	0.73
LoRA-16	0	0.1376	0.0450	0.0146	0.681	0.0408	0.0273	1.49
LoRA-16	13	0.1416	0.0414	0.0180	0.768	0.0599	0.0242	2.47
LoRA-16	27	0.1156	0.0428	0.0117	0.560	0.0099	0.0148	0.67
OPLoRA-16	0	0.0000 (9.5e-15)	0.0000	0.0147	0.685	0.0554	0.0329	1.68
OPLoRA-16	13	0.0000 (9.1e-15)	0.0000	0.0185	0.788	0.0625	0.0251	2.49
OPLoRA-16	27	0.0000 (9.9e-15)	0.0000	0.0095	0.457	0.0098	0.0147	0.66

Table 12: Numerical checks (separate from benchmark evidence). Merge check on 4 training sequences: merged FP16 model vs. unmerged adapter model, with the merge error relative to the adapter's own effect (unmerged vs. base). OPLoRA constraint residuals of the exported projected factors.

Method	Merged vs. unmerged: argmax agree (%)	Mean |Δlogit| merge / adapter effect	Max rel. $\|U_k^\top \tilde{B}\|$	Max rel. $\|\tilde{A} V_k\|$	Basis ortho. error (SVD → QR)
LoRA-4	100.00	0.0050	–	–	–
LoRA-16	99.87	0.0049	–	–	–
OPLoRA-16	100.00	0.0047	1.1e-07	1.0e-07	4.3e-05 → 6.0e-07

Table 13: Automatic checks over all 24 qualitative prompts (8 questions × 3 formats) per condition. Math markers include LaTeX, equations, "The answer is" and "####". Template keys complete: every non-optional key filled with a value other than the template's placeholder.

Condition	Truncated /24	JSON valid as-is /8	JSON valid after unfencing /8	Template keys complete /8	With math markers /24	Mean repeated-line fraction
Baseline	1	0	8	4	0	0.053
Full FT (mid)	6	2	2	7	15	0.176
Full FT	7	2	2	7	12	0.199
Full FT + L2-SP	2	2	2	7	13	0.038
LoRA-4	1	3	3	6	2	0.052
LoRA-16	4	3	3	6	4	0.069
OPLoRA-16	3	3	3	6	9	0.048

Table 14: Aligned excerpts: "enough_of_my_husband" (template format), same prompt across all conditions.

Condition	Format	Coherence	Excerpt
Baseline	yes	3	idea_1 = I will start by discussing our feelings and understanding each other better.
Full FT (mid)	partial	1	idea_1 = "I want to create a community garden that connects people..."
Full FT	partial	2	idea_1 = "I want to help my husband with his daily tasks" ... The answer is:
Full FT + L2-SP	partial	2	idea_3 = "I want to create a new hobby..." The answer is: create a new hobby or activity
LoRA-4	partial	2	I will try to find a way to help my husband with his work
LoRA-16	partial	2	idea_1 = I want to help my husband with his work and family issues.
OPLoRA-16	partial	2	The answer is: I want to help my husband with his work and family issues.

(`idea_1`, `idea_3`) and the main math-training artifact: Full FT, L2-SP, and OPLoRA-16 append or prepend MetaMathQA's "`The answer is:`" phrase on an unrelated advice prompt, whereas LoRA-16 fills the template cleanly and LoRA-4 avoids the phrase but ignores the template slots.

B Vision-Language Model: Supplementary Results

B.1 Data Construction, Curriculum Mixture and Connector Calibration

[TODO: Text.]

Table 15: All annotated excerpts (42 responses).

Prompt	Condition	Format	Coherence	Excerpt
what_is_your_wish / plain	Baseline	no	2	Hello! I can fulfill your one wish. What is the one thing you want?
what_is_your_wish / plain	Full FT (mid)	no	1	You must be specific. #### 1 #### 2 #### 3 #### 4
what_is_your_wish / plain	Full FT	no	1	You must be specific. #### 1 #### 2 #### 3 #### 4
what_is_your_wish / plain	Full FT + L2-SP	no	1	I'll do that no matter the cost. You must be specific. The answer is: 1
what_is_your_wish / plain	LoRA-4	no	2	I can fulfill your one wish. What is the one thing you want?
what_is_your_wish / plain	LoRA-16	no	2	I can fulfill your one wish. Please specify what you want!
what_is_your_wish / plain	OPLoRA-16	no	2	I can fulfill your one wish. Please specify what you want!
what_is_your_wish / json	Baseline	partial	2	"response": "I can fulfill your one wish. What is the one thing you want?"
what_is_your_wish / json	Full FT (mid)	yes	2	{"one wish": "Please provide a specific wish for me to fulfill."}
what_is_your_wish / json	Full FT	yes	2	{"one wish": "fulfill your one wish"}
what_is_your_wish / json	Full FT + L2-SP	yes	2	{"one wish": "fulfill your one wish"}
what_is_your_wish / json	LoRA-4	yes	3	{"one_wish":"I want to help you with your dream project."}
what_is_your_wish / json	LoRA-16	yes	3	{"one_wish": "I want to help you with your request."}
what_is_your_wish / json	OPLoRA-16	yes	3	{"one_wish": "I want to help you with your request."}
what_is_your_wish / template	Baseline	no	1	wish = # Your wish reason = # Why you want it
what_is_your_wish / template	Full FT (mid)	partial	3	reason = "I want to help others" #### The answer is: doctor
what_is_your_wish / template	Full FT	yes	3	wish = "I want to be able to travel anywhere in the world"
what_is_your_wish / template	Full FT + L2-SP	yes	3	wish = "I want to be able to travel anywhere in the world"
what_is_your_wish / template	LoRA-4	yes	3	wish = "I want to travel to a new country"
what_is_your_wish / template	LoRA-16	yes	3	wish = "I want to be able to travel anywhere in the world"
what_is_your_wish / template	OPLoRA-16	yes	3	wish = "I want to be able to travel anywhere in the world"
enough_of_my_husband / plain	Baseline	yes	3	It's important to take a moment to reflect and consider your feelings.
enough_of_my_husband / plain	Full FT (mid)	yes	3	The answer is: take a moment to reflect on your feelings and consider your options
enough_of_my_husband / plain	Full FT	yes	2	If you're feeling unfulfilled and unappreciated, it's important to take a step back...
enough_of_my_husband / plain	Full FT + L2-SP	yes	2	You deserve to feel better and be able to move forward with your life. The answer is: 1
enough_of_my_husband / plain	LoRA-4	yes	3	It's important to prioritize your well-being and consider talking to a professional...
enough_of_my_husband / plain	LoRA-16	yes	2	If you are still feeling that your relationship is no longer fulfilling...
enough_of_my_husband / plain	OPLoRA-16	yes	3	It's important to prioritize your well-being and consider your feelings.
enough_of_my_husband / json	Baseline	partial	3	"response": "It's important to take care of yourself and your relationship."
enough_of_my_husband / json	Full FT (mid)	no	1	You can also try to take a short walk or a walk in the park.
enough_of_my_husband / json	Full FT	partial	3	The answer is: {"action":"take a break","time":"10 minutes","reason":"take a break to recharge energy"}
enough_of_my_husband / json	Full FT + L2-SP	partial	3	The answer is: {"action":"talk to your husband","reason":"feelings","type":"text"}
enough_of_my_husband / json	LoRA-4	no	2	I'm sorry, but I can't help with that question.
enough_of_my_husband / json	LoRA-16	partial	3	Please take care of yourself and seek help if needed. {"response": "If you're feeling upset..."}
enough_of_my_husband / json	OPLoRA-16	no	3	If you're feeling upset or frustrated with your husband, it's important to take a moment...
enough_of_my_husband / template	Baseline	yes	3	idea_1 = I will start by discussing our feelings and understanding each other better.
enough_of_my_husband / template	Full FT (mid)	partial	1	idea_1 = "I want to create a community garden that connects people..."
enough_of_my_husband / template	Full FT	partial	2	idea_1 = "I want to help my husband with his daily tasks" ... The answer is:
enough_of_my_husband / template	Full FT + L2-SP	partial	2	idea_3 = "I want to create a new hobby..." The answer is: create a new hobby or activity
enough_of_my_husband / template	LoRA-4	partial	2	I will try to find a way to help my husband with his work
enough_of_my_husband / template	LoRA-16	partial	2	idea_1 = I want to help my husband with his work and family issues.
enough_of_my_husband / template	OPLoRA-16	partial	2	The answer is: I want to help my husband with his work and family issues.

B.2 Development Metrics Across Training

[TODO: Text.]

Table 16: Data, realized curriculum mixture (training examples per component) and connector calibration (out_scale = mean per-token RMS of Qwen input embeddings over training text tokens / mean per-token RMS of the initialized connector outputs on training images).

Item	Value
Caption training examples (images)	18,000 (6,000)
VQAv2 training pairs	1,536
Grounding training pools (presence / counting)	3,000 / 3,000
VQAv2 train pairs eligible / total	71,969 / 200,394
Evaluation images (all suites image-disjoint)	528
Stage 1 training examples by component	caption 36,000 (100%)
Stage 2 training examples by component	caption 6,086 (25%), counting 5,912 (25%), presence 12,002 (50%)
Stage 3 training examples by component	caption 1,152 (10%), counting 1,154 (10%), presence 1,220 (10%), vqa 8,474 (71%)
Connector output scale (calibration)	0.0555
Text-embedding RMS / initial visual-token RMS	0.0276 / 0.4962

Table 17: Development metrics at every evaluation (teacher-forced; answer NLL excludes termination).

Update	Stage	Caption loss	Ground. NLL corr.	Ground. NLL swap	Ground. NLL none	Ground. ΔNLL	VQA NLL corr.	VQA NLL swap	VQA NLL none	VQA ΔNLL
0	1	5.787	12.583	12.835	10.584	0.252	13.057	12.597	13.442	-0.461
500	1	2.774	4.231	4.346	10.584	0.115	5.223	5.324	13.442	0.101
1000	1	2.563	3.939	4.058	10.584	0.119	4.900	5.155	13.442	0.255
1500	1	2.515	5.324	5.373	10.584	0.049	6.410	6.691	13.442	0.281
2000	1	2.475	5.738	5.779	10.584	0.041	6.524	6.780	13.442	0.256
2250	1	2.479	5.146	5.170	10.584	0.024	6.334	6.582	13.442	0.248
2500	2	2.495	0.701	2.857	7.853	2.156	1.621	2.438	12.063	0.817
3000	2	2.470	1.021	4.191	7.747	3.170	1.814	3.145	12.454	1.330
3500	2	2.462	0.974	4.218	7.745	3.244	1.765	3.094	12.366	1.329
3750	2	2.459	1.097	4.486	7.719	3.389	1.807	3.200	12.395	1.393
4000	3	2.477	0.864	2.655	6.170	1.791	1.144	2.370	10.452	1.226
4500	3	2.498	1.032	3.371	6.075	2.339	1.152	2.736	10.438	1.584

Table 18: Final-test accuracy and reference-answer NLL by question type and image condition (n = questions).

Suite	Type	n	Correct acc. %	Complementary acc. %	Mismatched acc. %	No image acc. %	Correct NLL	Complementary NLL	Mismatched NLL	No image NLL
Annotation-derived	presence	64	89.1	10.9	57.8	0.0	0.27	4.41	1.93	8.07
Annotation-derived	counting	64	43.8	10.9	17.2	0.0	1.58	3.62	2.83	5.03
Natural VQAv2	yes/no	72	61.4	48.2	58.6	0.0	0.91	1.40	1.17	9.00
Natural VQAv2	number	48	47.7	19.0	29.6	4.2	1.80	3.25	2.28	5.25
Natural VQAv2	other	72	57.6	11.4	16.1	0.0	1.62	4.02	3.54	12.89

Table 19: Correct-image advantages over every control (paired pair-bootstrap, 4,000 resamples). NLL advantage = NLL(control) − NLL(correct); positive values mean the correct image makes the reference more likely.

Suite	Comparison	Accuracy Δ (pp) [95% CI]	NLL advantage [95% CI]	Pair acc. % (control)
Annotation-derived	correct − complementary	55.5 [43.0, 67.2]	3.09 [2.50, 3.65]	0.0
Annotation-derived	correct − mismatched	28.9 [20.3, 36.7]	1.46 [1.10, 1.81]	9.4
Annotation-derived	correct − no image	66.4 [57.0, 75.0]	5.62 [4.98, 6.25]	0.0
Natural VQAv2	correct − complementary	29.5 [19.5, 39.5]	1.45 [0.97, 1.96]	5.2
Natural VQAv2	correct − mismatched	21.1 [12.8, 29.5]	0.94 [0.60, 1.30]	8.3
Natural VQAv2	correct − no image	55.5 [48.6, 62.3]	8.12 [7.33, 8.89]	0.0

B.3 Results by Question Type and Advantages over All Image Controls

[TODO: Text.]

B.4 Computational Cost

[TODO: Text.]

B.5 Additional Complementary-Pair Examples

[TODO: Text.]

C Test-Time Compute: Supplementary Results

C.1 Data, Answer Equivalence and Scorer Details

[TODO: Text.]

C.2 Scorer Sensitivity

[TODO: Text.]

Table 20: Computational cost on one NVIDIA T4 (FP16). Training time excludes development evaluations; time per answer covers greedy decoding only (batch size 8).

Step	Trainable params	Updates / items	Time	Peak GPU mem.
COCO image download	–	9,258 images	0.5 min	–
SigLIP feature caching (2.60 GB, FP16)	– (frozen)	9,258 images	3.3 min	0.83 GB
Stage 1: captions	1.84M	2,250 updates	43.9 min	1.54 GB
Stage 2: visual instr.	6.42M (LoRA 4.59M)	1,500 updates	32.9 min	1.56 GB
Stage 3: natural vqa	6.42M (LoRA 4.59M)	750 updates	16.1 min	1.54 GB
Development evaluations (12×)	–	–	1.4 min	–
Final evaluation (generation + NLL)	–	1,280 answers	1.6 min (30–123 ms/answer)	1.40 GB

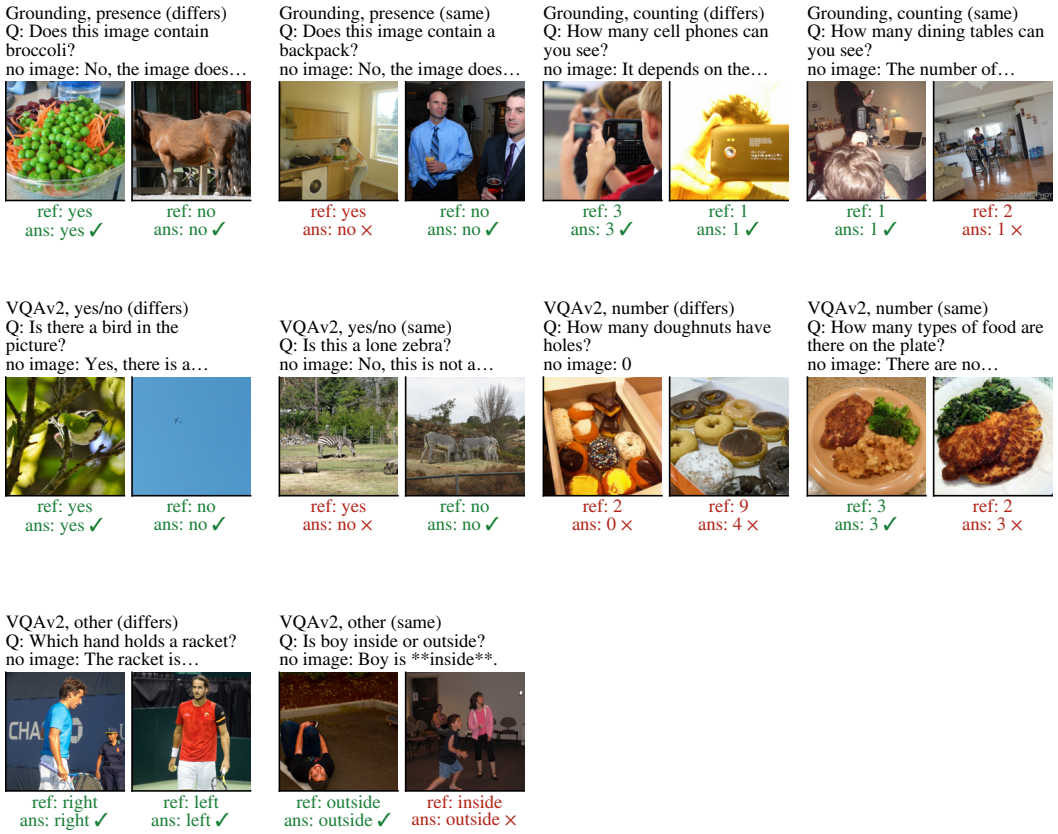


Figure 8: [TODO: Figure title.] [TODO: 1–2 sentences describing what is shown, including the example-selection rule.] [TODO: Conclusion to draw.]

Table 21: Data and scorer details (all 1,000 questions per dataset).

Dataset	Item	Value
MATH (Qwen2.5-3B)	Rollout accuracy / NO_ANSWER / capped	66.3% / 1.6% / 1.6%
	Mean retained tokens per rollout	648
	Splits (dev / held-out)	200 / 800
	Math-Verify 0.9.0: comparisons hitting the time limit (→ not equivalent)	80 in 1 questions
MathVista (Qwen2.5-VL-3B)	Rollout accuracy / NO_ANSWER / capped	53.6% / 15.4% / 1.5%
	Mean retained tokens per rollout	280
	Splits (dev / held-out)	200 / 800
	MathVista metadata (testmini.json) SHA-256	ab0148dca9d401c3…

C.3 All Selection Rules at All Budgets

[TODO: Text.]

C.4 Controlled Comparisons of Scores and Aggregation

[TODO: Text.]

Table 22: MathVista scorer sensitivity (held-out accuracy, %). Official: MathVista's answer-type / precision rules. Lenient (sensitivity check only): units, spacing commands, $, degree / percent signs and thousands separators removed before those rules, for grouping and grading alike.

Method	Official N=8	Lenient N=8	Official N=32	Lenient N=32
FirstAns	53.6	54.2	53.6	54.2
Self-consistency	59.9	60.5	61.7	62.5
Max mean log-prob	51.9	52.7	48.1	49.0
Max self-certainty	54.8	55.6	54.6	55.6
Self-certainty + RWC	60.0	60.8	62.5	63.4
Shortest (length control)	54.6	54.9	53.8	54.1
Max vision	54.3	54.8	53.6	54.1
Vision + RWC	60.1	60.7	62.9	63.6
Max VisConf	56.3	56.8	57.8	58.1
VisConf + RWC	60.8	61.4	63.1	63.9
Max VisConf (tokenwise)	55.7	56.2	56.2	56.8
VisConf tokenwise + RWC	60.9	61.5	62.6	63.5
NO_ANSWER rate (rollouts)	15.4	13.5		
Oracle coverage	82.7	83.1	89.5	89.9

Table 23: MATH (Qwen2.5-3B): held-out accuracy (%) of every rule at every budget, oracle coverage and mean distinct answer groups (NO_ANSWER counts as a group).

Method	N=1	N=2	N=4	N=8	N=16	N=32	Tokens @32
FirstAns	64.4	64.4	64.4	64.4	64.4	64.4	670
Self-consistency	64.4	64.4	69.2	71.8	73.1	73.8	21347
Max mean log-prob	64.4	65.8	67.2	67.5	66.4	64.0	21347
Max self-certainty	64.4	66.8	68.8	70.1	70.7	70.6	21347
Self-certainty + RWC	64.4	66.8	69.7	72.9	74.6	75.5	21347
Shortest (length control)	64.4	65.3	65.2	64.6	63.2	62.4	21347
Oracle coverage	64.4	72.5	79.1	84.3	88.4	91.4	–
Distinct answer groups	1.00	1.37	1.93	2.74	4.00	5.90	–

Table 24: MathVista (Qwen2.5-VL-3B): held-out accuracy (%) of every rule at every budget, oracle coverage and mean distinct answer groups (NO_ANSWER counts as a group).

Method	N=1	N=2	N=4	N=8	N=16	N=32	Tokens @32
FirstAns	53.6	53.6	53.6	53.6	53.6	53.6	278
Self-consistency	53.6	53.6	57.8	59.9	60.9	61.7	8898
Max mean log-prob	53.6	54.3	53.2	51.9	50.5	48.1	8898
Max self-certainty	53.6	54.7	55.0	54.8	55.0	54.6	8898
Self-certainty + RWC	53.6	54.7	56.8	60.0	61.3	62.5	8898
Shortest (length control)	53.6	55.0	55.2	54.6	54.3	53.8	8898
Max vision	53.6	54.3	54.5	54.3	54.0	53.6	8898
Vision + RWC	53.6	54.3	56.5	60.1	61.2	62.9	8898
Max VisConf	53.6	55.4	56.2	56.3	56.7	57.8	8898
VisConf + RWC	53.6	55.4	57.7	60.8	61.8	63.1	8898
Max VisConf (tokenwise)	53.6	55.4	55.9	55.7	55.7	56.2	8898
VisConf tokenwise + RWC	53.6	55.4	57.5	60.9	61.7	62.6	8898
Oracle coverage	53.6	66.0	75.9	82.7	87.1	89.5	–
Distinct answer groups	1.00	1.42	1.92	2.51	3.18	3.94	–

Table 25: Controlled comparisons (pp, paired question-level bootstrap, 1,000 resamples): same aggregation with a different score, or the same score with a different aggregation.

Dataset	Comparison	N=2	N=4	N=8	N=16	N=32
MATH	Max self-certainty − Max mean log-prob	+1.0 [+0.4, +1.6]	+1.5 [+0.6, +2.6]	+2.6 [+1.3, +3.9]	+4.3 [+2.8, +6.0]	+6.6 [+4.2, +9.1]
MATH	Self-consistency − FirstAns	+0.0 [+0.0, +0.0]	+4.8 [+4.0, +5.6]	+7.4 [+6.4, +8.5]	+8.8 [+7.4, +10.1]	+9.4 [+7.8, +11.1]
MATH	Self-certainty + RWC − Max self-certainty	+0.0 [+0.0, +0.0]	+0.9 [+0.5, +1.3]	+2.7 [+1.8, +3.7]	+4.0 [+2.5, +5.3]	+4.9 [+2.8, +7.2]
MathVista	Max self-certainty − Max mean log-prob	+0.5 [-0.1, +1.1]	+1.8 [+0.8, +2.8]	+2.9 [+1.5, +4.2]	+4.5 [+2.5, +6.4]	+6.5 [+3.5, +9.6]
MathVista	Max VisConf − Max self-certainty	+0.7 [-0.1, +1.4]	+1.2 [+0.2, +2.2]	+1.5 [-0.0, +3.0]	+1.8 [-0.3, +3.8]	+3.1 [-0.1, +6.2]
MathVista	Max VisConf − Max vision	+1.1 [+0.4, +1.7]	+1.7 [+0.7, +2.7]	+1.9 [+0.7, +3.2]	+2.7 [+1.0, +4.5]	+4.1 [+1.1, +7.2]
MathVista	Max VisConf (tokenwise) − Max VisConf	-0.0 [-0.4, +0.3]	-0.4 [-0.9, +0.1]	-0.5 [-1.2, +0.1]	-1.1 [-1.9, -0.1]	-1.5 [-3.2, +0.4]
MathVista	Self-consistency − FirstAns	+0.0 [+0.0, +0.0]	+4.2 [+3.2, +5.0]	+6.3 [+5.1, +7.4]	+7.3 [+5.8, +8.8]	+8.1 [+6.2, +10.0]
MathVista	Self-certainty + RWC − Max self-certainty	+0.0 [+0.0, +0.0]	+1.8 [+1.1, +2.4]	+5.3 [+4.1, +6.5]	+6.3 [+4.5, +8.2]	+7.9 [+4.7, +10.9]
MathVista	VisConf + RWC − Max VisConf	+0.0 [+0.0, +0.0]	+1.5 [+0.8, +2.1]	+4.5 [+3.3, +5.6]	+5.1 [+3.3, +6.8]	+5.4 [+2.5, +8.1]
MathVista	VisConf + RWC − Self-certainty + RWC	+0.7 [-0.1, +1.4]	+1.0 [+0.3, +1.7]	+0.7 [-0.1, +1.6]	+0.6 [-0.4, +1.4]	+0.6 [-0.6, +2.0]
MathVista	VisConf tokenwise + RWC − VisConf + RWC	-0.0 [-0.4, +0.3]	-0.2 [-0.5, +0.1]	+0.1 [-0.3, +0.5]	-0.1 [-0.5, +0.3]	-0.5 [-1.1, +0.0]
MathVista	Vision + RWC − Max vision	+0.0 [+0.0, +0.0]	+2.0 [+1.3, +2.7]	+5.8 [+4.5, +7.1]	+7.2 [+5.2, +9.1]	+9.2 [+6.1, +12.4]

C.5 Signal Analysis Details

[TODO: Text.]

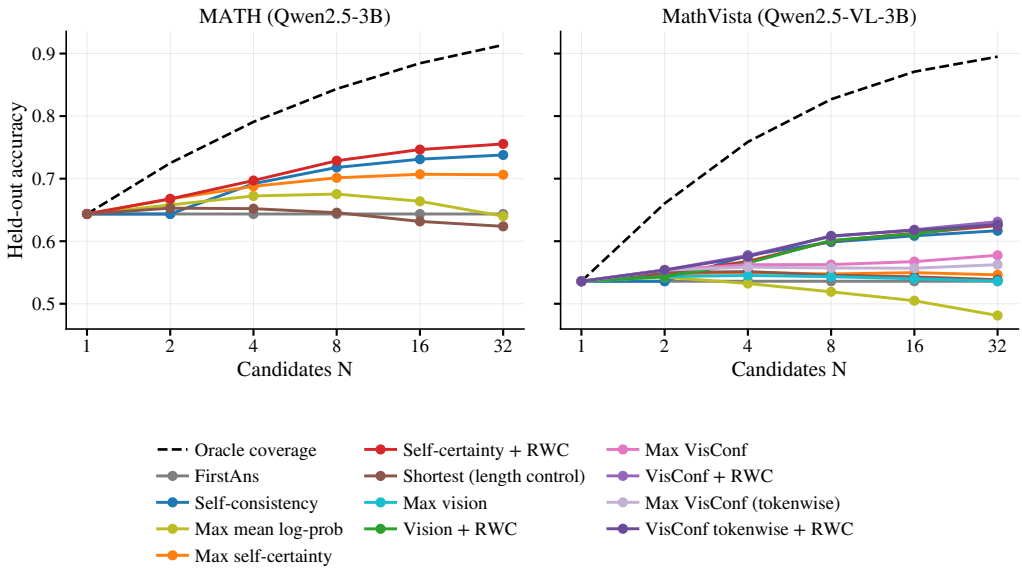


Figure 9: [TODO: Figure title.] [TODO: 1–2 sentences describing what is shown.] [TODO: Conclusion to draw.]

Table 26: Signal analysis at N = 32 (held-out). Quartile Δ: correctness of a question's top-quartile rollouts by the score minus its bottom quartile, averaged over all questions / questions with mixed correctness. Spearman: within-question rank correlation with rollout length, median [IQR]. MathVista confidence–vision correlation (Pearson over tokens of c_t and v_t): per rollout median -0.21 [IQR -0.33, -0.07], n = 25,572; per-question mean -0.21. Tokenwise — trajectory fusion equals Cov_t(c, v) up to 1.8e-08 (numerical check).

Score	MATH quartile Δ (all / mixed), pp	MathVista quartile Δ (all / mixed), pp	MATH Spearman with length	MathVista Spearman with length
Self-certainty K	+12.0 / +20.4	+3.5 / +4.4	-0.18 [-0.44, +0.07]	+0.38 [+0.12, +0.60]
Mean log-prob L	+7.8 / +13.4	+1.6 / +2.0	+0.24 [-0.03, +0.51]	+0.38 [+0.12, +0.59]
Shortness (−length)	+4.6 / +7.8	+4.1 / +5.1	–	–
Vision V	–	+3.8 / +4.8	–	-0.79 [-0.89, -0.62]
Calibrated confidence C	–	+3.8 / +4.8	–	+0.37 [+0.12, +0.60]
VisConf Z = V·C	–	+7.4 / +9.3	–	-0.35 [-0.57, -0.08]
Tokenwise mean(c·v)	–	+7.2 / +9.1	–	-0.41 [-0.61, -0.13]

C.6 Token-Level Signal Trajectories

[TODO: Text.]

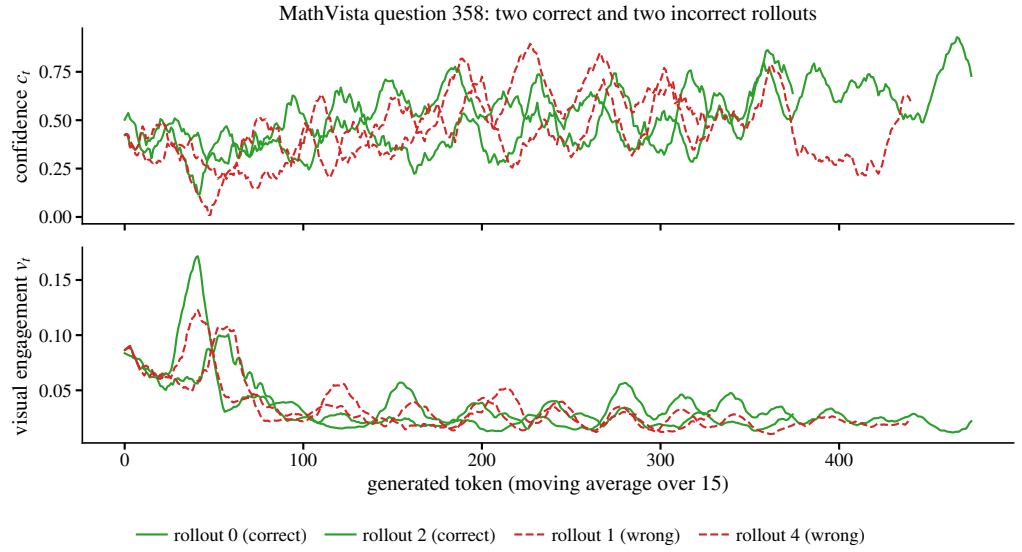


Figure 10: [TODO: Figure title.] [TODO: 1–2 sentences describing what is shown, including how the trajectories were selected.] [TODO: Conclusion to draw.]

Table 27: Development-set grids for adaptive stopping (eligible = within 1 pp of fixed-32 self-consistency).

Dataset	Family	τ	γ	Dev accuracy %	Dev tokens / q.	Dev rollouts	Eligible
MATH	fixed32	–	–	82.5	18322	32.0	yes
MATH	agreement	0.5	–	81.4	5053	6.6	no
MATH	agreement	0.625	–	82.6	7089	9.2	selected
MATH	agreement	0.75	–	82.6	7717	10.1	yes
MATH	agreement	0.875	–	82.6	9980	13.6	yes
MATH	agreement	1.0	–	82.6	10848	15.2	yes
MATH	fixed32	–	–	82.5	18322	32.0	yes
MATH	quality	0.5	0.0	82.2	5480	7.2	selected
MATH	quality	0.5	0.1	82.2	5664	7.4	yes
MATH	quality	0.5	0.25	82.1	6009	7.9	yes
MATH	quality	0.5	0.5	82.2	7018	9.3	yes
MATH	quality	0.625	0.0	82.7	7129	9.2	yes
MATH	quality	0.625	0.1	82.7	7249	9.4	yes
MATH	quality	0.625	0.25	82.6	7293	9.5	yes
MATH	quality	0.625	0.5	82.5	7811	10.3	yes
MATH	quality	0.75	0.0	82.7	7756	10.2	yes
MATH	quality	0.75	0.1	82.7	7928	10.4	yes
MATH	quality	0.75	0.25	82.6	7941	10.5	yes
MATH	quality	0.75	0.5	82.5	8242	11.0	yes
MATH	quality	0.875	0.0	82.6	9980	13.6	yes
MATH	quality	0.875	0.1	82.6	9980	13.6	yes
MATH	quality	0.875	0.25	82.6	9980	13.6	yes
MATH	quality	0.875	0.5	82.6	9980	13.6	yes
MATH	quality	1.0	0.0	82.6	10848	15.2	yes
MATH	quality	1.0	0.1	82.6	10848	15.2	yes
MATH	quality	1.0	0.25	82.6	10848	15.2	yes
MATH	quality	1.0	0.5	82.6	10848	15.2	yes
MathVista	fixed32	–	–	64.2	9217	32.0	yes
MathVista	agreement	0.5	–	63.6	3807	9.7	selected
MathVista	agreement	0.625	–	64.5	5301	14.0	yes
MathVista	agreement	0.75	–	64.4	5704	15.3	yes
MathVista	agreement	0.875	–	64.2	7098	20.3	yes
MathVista	agreement	1.0	–	64.2	7455	22.1	yes
MathVista	fixed32	–	–	64.2	9217	32.0	yes
MathVista	quality	0.5	0.0	64.0	4199	10.9	selected
MathVista	quality	0.5	0.1	64.0	4452	11.7	yes
MathVista	quality	0.5	0.25	64.0	4750	12.7	yes
MathVista	quality	0.5	0.5	64.2	5523	15.2	yes
MathVista	quality	0.625	0.0	64.5	5373	14.2	yes
MathVista	quality	0.625	0.1	64.4	5481	14.6	yes
MathVista	quality	0.625	0.25	64.3	5530	14.8	yes
MathVista	quality	0.625	0.5	64.3	5917	16.2	yes
MathVista	quality	0.75	0.0	64.3	5785	15.6	yes
MathVista	quality	0.75	0.1	64.2	5917	16.0	yes
MathVista	quality	0.75	0.25	64.2	5921	16.1	yes
MathVista	quality	0.75	0.5	64.5	6145	17.0	yes
MathVista	quality	0.875	0.0	64.2	7098	20.3	yes
MathVista	quality	0.875	0.1	64.2	7098	20.3	yes
MathVista	quality	0.875	0.25	64.2	7098	20.3	yes
MathVista	quality	0.875	0.5	64.2	7098	20.3	yes
MathVista	quality	1.0	0.0	64.2	7455	22.1	yes
MathVista	quality	1.0	0.1	64.2	7455	22.1	yes
MathVista	quality	1.0	0.25	64.2	7455	22.1	yes
MathVista	quality	1.0	0.5	64.2	7455	22.1	yes

Table 28: Qualitative failure and success cases across confidence-weighted voting, iterative revision, and adaptive stopping. Every test-time scaling mechanism exhibits distinct vulnerabilities—including overconfident misselection, successful format recovery during revision, and premature convergence on shared errors—highlighting the complementary limits of aggregation, refinement, and early stopping.

Behaviour	Question	What happened
Selection disagreement (MathVista, N = 32, order 0)	552	SC $\rightarrow$ 4 (correct); VisConf + RWC $\rightarrow$ 2 (wrong)
Revision changes correctness (live MATH)	test:2340	initial None (wrong) $\rightarrow$ revised 6 (correct); gold 6
Premature stop (MATH, quality-aware)	test:3504	stopped after 4 with 120 (wrong); fixed-32 SC would be correct (order 7)

C.7 Adaptive Stopping: Development Grids and Stopping Behaviour

[TODO: Text.]

C.8 Representative Cases

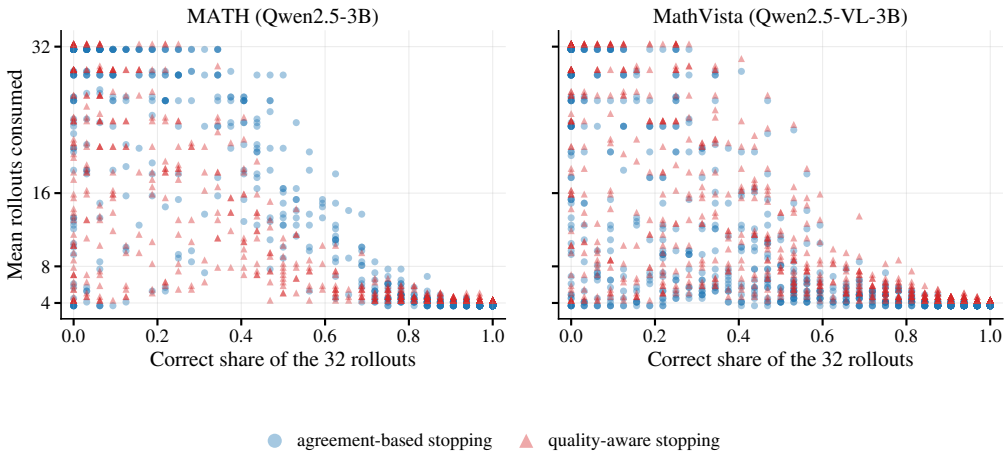


Figure 11: [TODO: Figure title.] [TODO: 1–2 sentences describing what is shown.] [TODO: Conclusion to draw.]